

What the Theory Predicts, and What the Data Actually Rewards

Graph connectivity as a probe of transformer reasoning — progress report

Barsbold Bayarerdene

August 28, 2026

Diploma research

Done. Reproduced Yehudai's connectivity experiment; read and annotated all three papers; built a leak-audited dataset; ran 150 experiments.

The finding that reorganised the project. Every benchmark I touched — including two from published papers — turned out to be solvable by a statistic that has nothing to do with connectivity.

What I need guidance on. There is a gap between what the theory says a transformer **can represent** and what my training runs actually **find**. I am not sure which side of it to make the thesis.

Part 1 — What the papers claim

Depth-Width Tradeoffs for Transformers on Graph Tasks (NeurIPS 2025). A plain transformer encoder, no message passing; the graph is flattened into tokens.

tokenization	one token =	seq len
adj_rows	node's adjacency row $\oplus$ features	n
edge_list	$\text{onehot}(u) \oplus x_u \parallel \text{onehot}(v) \oplus x_v$	#edges
lap_full	Laplacian eigenvectors $\oplus$ features	n

Prior work fixed sub-linear width and asked for depth. They invert it: with **linear** width in n , **constant** depth suffices — including for **graph connectivity**.

Clayton Sanford co-authors both this and the next paper — a follow-up, not a rival.

The width hierarchy, and how width buys connectivity

task	width at fixed depth	result
node degree	constant	local, trivial
2-cycles, bounded degree d	$O(d \log n)$	Thm 4.4, optimal up to logs
1 vs. 2 cycles	$O(n)$, 2 layers	Thm 4.1
connectivity	$O(n \text{ polylog } n)$	§4.1, proof sketch only
Eulerian cycle verification	near-quadratic	Thm 5.1, conditional

Connectivity via **linear sketching** (Ahn et al. 2012): compress each adjacency row to $O(\text{polylog } n)$, pack all tokens into one, let the MLP finish. High-probability, not deterministic. Alternative route — Thm 4.3: depth $O(L)$, $m = O(n)$ computes A^L .

Thm 4.2 ($mpHL = \Omega(n)$ for 2-cycles on adjacency rows) is **unconditional**.

Their empirical claim

Fixed budget (100k params), $n = 100$, adjacency rows, splitting the budget as (1,125), (2,89), (4,63), (8,45), (10,40):

- loss and test accuracy **consistent across every configuration**
- shallow-and-wide **significantly faster** to train and to run

Their headline is not “wide is more accurate” — it is “**wide is equally accurate and much cheaper**”.

Note the discipline: a **parameter-matched** sweep. My own depth×width grid lacks exactly this, and I come back to it at the end.

Understanding Transformer Reasoning Capabilities via Graph Algorithms (NeurIPS 2024).

regime	depth / width	what lives here
D1	$L = 1, m = O(\log N)$	retrieval
LogDepth	$L = O(\log N), m = O(N^\varepsilon)$	connectivity
LogDepthWide	$L = O(\log N), m = O(N^{\frac{1}{2}+\varepsilon})$	shortest path

Theorem 1 is the engine. Any R -round MPC protocol with $O(N^\delta)$ local memory is simulated by a transformer of depth $L = O(R)$, width $m = O(N^{\delta+\varepsilon})$ — MLPs do local computation, one attention layer does one round of communication.

So tasks sort by **round complexity**, not apparent difficulty.

N is the input **sequence length** $O(|V| + |E|)$, not the node count. $\varepsilon > 0$ is any fixed constant.

What Sanford says about connectivity

result	content
Theorem 2	connectivity is in LogDepth — the construction exists
Theorem 3	log depth is necessary (conditional on their Conjecture 13)
Theorem 6	one layer provably cannot do it without width $mH = \tilde{\Omega}(N)$

§3.4 is the one exception: depth drops to $O(\log \log N)$ — but only with $O(\alpha N^{1-\varepsilon})$ **pause tokens**.

Depth goes below logarithmic **only** when the computation is moved onto the tape. Hold that thought — it is the hinge of this presentation.

Transformers Provably Learn Algorithmic Solutions for Graph Connectivity, But Only with the Right Data (ICML 2026). A depth- L model has reachability capacity 3^L ; their lever is to train only on graphs **within** capacity, starving the heuristic channel.

paper	fixes	connectivity verdict
Sanford 2024	width $O(N^\epsilon)$	needs depth $O(\log N)$; 1 layer impossible
Yehudai 2025	depth $O(1)$	needs width $O(n \text{ polylog } n)$
Ye 2026	architecture	learnable only with the right training data

Connectivity is **expensive** — you pay in depth, or in width, or you do not get it. All three treat the resource as something the **model** must spend.

Part 2 — Reproducing Yehudai

It reproduces, and the control clears my pipeline

$n = 50$, 1 layer, width 64, 100 train graphs, 100 epochs:

model	data	test
theirs, adj_rows	Yehudai $n = 50$	1.000
theirs, edge_list	Yehudai $n = 50$	0.926
mine, 1 layer, 15,874 params	Yehudai $n = 50$	1.000 @ epoch 4
mine	my connectedness_hard	≈ 0.59

My pipeline beats their own implementation on their data — so everything that follows is **about the data**, not about my code.

Already suspicious: one layer solves connectivity at $n = 50$. If the task needed $O(\log N)$ depth, that should not happen.

Part 3 — The Laplacian result

The prediction, stated sharply

Both papers rule out the same corner, from opposite directions:

source	what it forbids
Sanford, Thm 6	one layer needs width $mH = \tilde{\Omega}(N)$
Yehudai, §4.1 + Thm 4.2	fixed depth needs width $O(n \text{ polylog } n)$

At $n \approx 20$ both say a one-layer model needs width on the order of n at minimum, and realistically in the hundreds.

So — one layer, width 8. What should happen?

What actually happens

connectedness_hard: degree- and edge-count-matched, classes differing by **one bridge edge**. Left, Laplacian as an add-on; right, Laplacian as the node features:

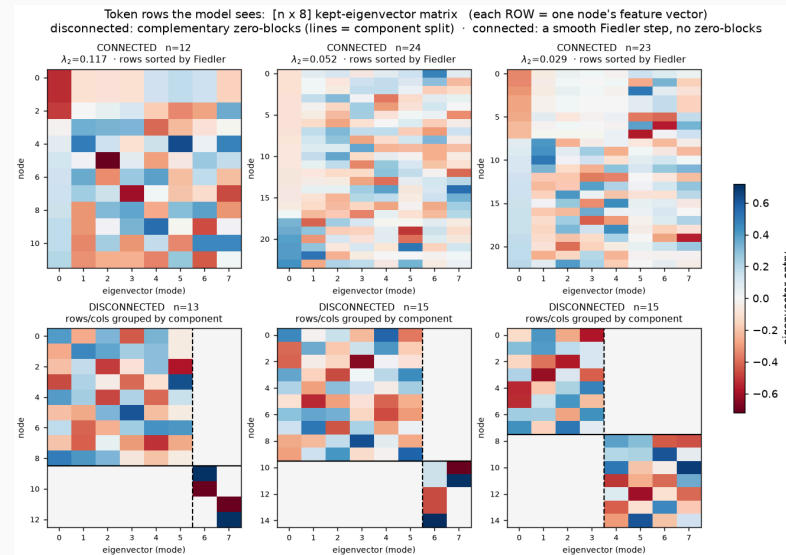
depth	width	LPE	test	width (LPE-as-features)	test
1	64	0	0.545	64	1.000
1	64	16	1.000	32	1.000
1	24	16	1.000	16	1.000
1	16	8	1.000	8	1.000
1	8	4	1.000	4	0.915

Depth 1, width 8, four Laplacian dimensions — perfect accuracy on the task a depth-8 model cannot beat 0.64 on without them. Degradation only begins at width 4.

results/20260619_221812_*, _222010_*, 20260621_011108_*, 20260623_110250_* ... _111551_*

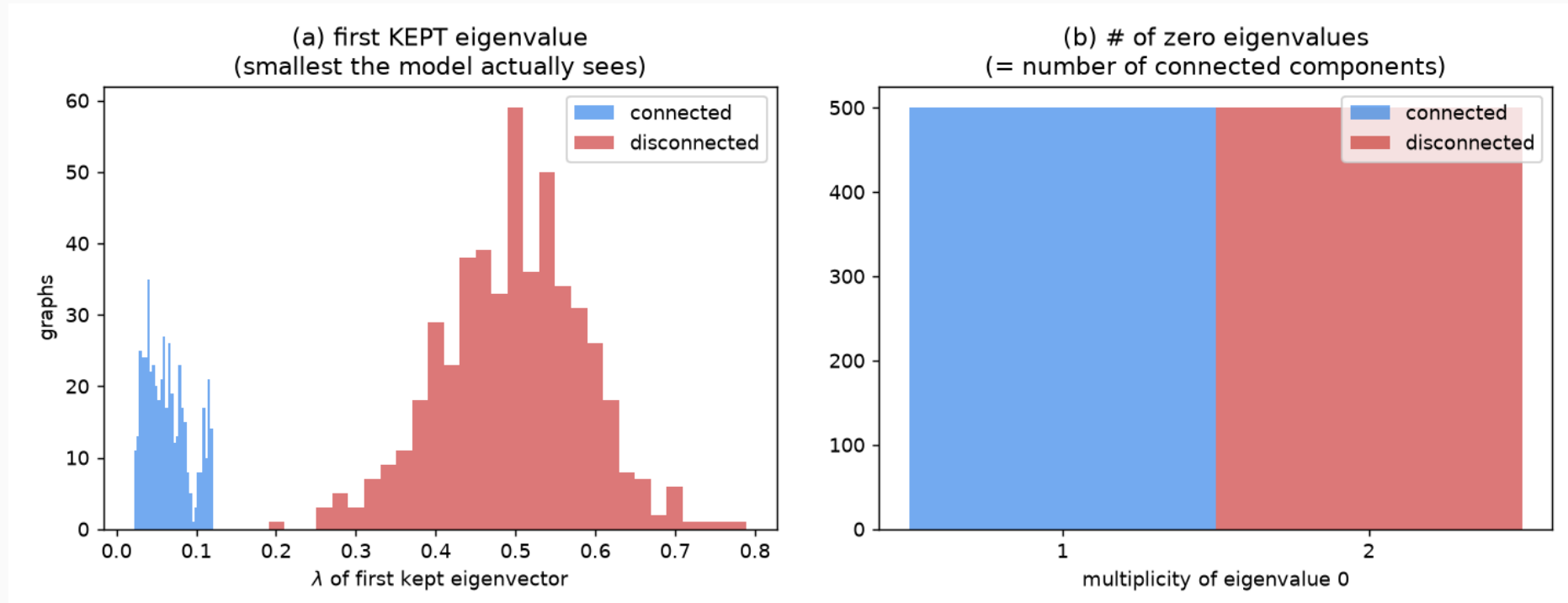
Why: the answer is already in the input

$L = D - A$ has one zero eigenvalue **per connected component**, and its zero-eigenvalue eigenvectors are **indicator vectors on those components**. So the encoding hands over the component count and the membership directly.



The $[n \times 8]$ matrix the model receives, one row per node. **Disconnected** graphs (bottom) show **complementary zero-blocks** **connected** graphs (top) have none.

And a single scalar separates the classes



Left: the smallest eigenvalue the model sees. Connected graphs sit at **0.03–0.12**, disconnected at **0.25–0.80** — almost no overlap, on the dataset whose classes differ by **one edge**. Right: multiplicity of eigenvalue 0 is **exactly** the number of components, 500 / 500 on each side.

I did not discover this. Yehudai et al., Appendix A:

“the tokenization **trivializes the connectivity task** because a graph is disconnected if and only if its second-smallest eigenvalue is zero; transformers with the node-adjacency tokenization require either depth $\Omega(\log n)$ or width $\Omega(n)$ to solve the same problem.”

They state it as **theory**, and their own encoding experiments (§6.3) are on molecular property prediction, **not on connectivity**. Mine is the **measurement**: it holds on an adversarial dataset, at 4 eigenvector dimensions and width 8, with a matched no-LPE control at 0.545.

A confirmed prediction is worth reporting — but it is a confirmation, and I present it as one.

It does not falsify the theorem — it relocates the work

Eigendecomposition is $O(n^3)$ of **preprocessing**. The $O(\log N)$ depth has not been avoided — it has been **paid for outside the model**. Same structure as Sanford's own §3.4 pause-token exception.

An encoding is not a neutral input format. It is an oracle that has already done part of the task:

encoding	what it hands over	depth needed
lap_full / LPE	transitive closure — the components themselves	1
SPD / distance bias	all-pairs distances	low
adj_rows	a consistent layout — only if n is fixed	depends
edge_list	nothing	$\ln 2$ plateau

Part 4 — Every benchmark leaked

The ER dataset I started with. connectedness samples near $p \approx \frac{\log(n)}{n}$, where disconnection is almost always caused by **isolated vertices**:

trivial rule	accuracy
majority class	0.679
connected $\iff$ min degree ≥ 1	0.982

My token serialization. `diameter_controlled` gave connected graphs exactly 23 edges and disconnected ones 22 — so the prompt was two tokens longer and the positional embedding read the answer off **sequence length**. Invisible to every GNN baseline.

Seed 42, 1000 graphs · CHANGELOG 06-19 · serialization leak fixed in ed47ddd

They designed against this. Appendix E: “to avoid a correlation between connectivity and edge probability as exists in Erdős–Rényi graphs”, they mix four generators — ER, random geometric, Barabási–Albert, and SBM.

It leaks anyway, because the **classes** remain unmatched within each generator:

single-scalar rule	on Yehudai's data	on mine
total edge count	0.77	0.52
mean degree	0.77	0.52
any isolated node?	0.51	0.51

Their dataset lets a model answer “**is it connected?**” by asking “**how dense is it?**”

Leak in Ye's setup, caught by an adversarial probe

Two probes: **two disconnected cliques**, and **the same two cliques plus one bridge**. Exact-match on the reachability matrix:

regime	train EM	ER test	2 cliques, disc.	+ bridge
raw	1.000	0.578	0.000	1.000
within-capacity	1.000	0.698	0.000	1.000

0.0 on disconnected and 1.0 on connected is the signature of a model that always answers “**everything is reachable**” — reading **density**, not connectivity. Their capacity lever does not change it.

Sanford's own validation (GraphQA) uses 5–20-node ER graphs — the family Leak 1 convicts — and his Appendix E concedes the graphs “do not resemble the large-diameter worst-case instance” his theorems are about.

The catalogue — including my own dataset

where	the shortcut	strength
my ER dataset	is there a degree-0 node?	0.982
Yehudai's benchmark	how many edges? / mean degree	0.77
Laplacian encoding	components are literally in the input	1.000
my serialization	prompt length differs by class	1.000
Ye's setup	always answer “reachable”; density heuristic	0.0 / 1.0
my connectedness_hard	local bridge-detection	0.975

The last row is mine and I have to own it: a 3-layer GIN with degree features scores **0.975** — at depth 3 on graphs of diameter 4–8, so it cannot be global reasoning. My construction killed min-degree and mean-degree, but not local bridge-detection.

results/20260624_183646_* · CHANGELOG 06-25

Part 5 — What survives the audit

With the leaks sealed, nothing learns the binary task

connectedness_hard, binary label, no Laplacian.

- 20 runs: depths 1–8, widths 2–128, two encodings, up to 500 epochs
- **generalization** fails, not capacity — same model, varying only training-set size: 10 graphs → 1.00, 100 → 0.76, full → 0.545

depth	width	features	test
1 – 5	2 – 128	degree	0.500 – 0.545
1	20 – 128	adj_rows	0.565 – 0.640
4 – 8	32 – 128	adj_rows	0.520 – 0.600

Two things did work — both by changing the supervision

target	result	supervision
binary “is it connected?”	stalls at ln 2	1 bit / graph
$n \times n$ reachability matrix	$\approx$ 0.97 EM	n^2 bits / graph
emit a full BFS trace	0.997	dense, arranged in time

The trace result is depth **2** and flat across diameters 2–18, versus 0.510 for a matched answer-only control on identical data.

Neither is a better architecture. Both are the same lever: **more supervision per graph**.

CHANGELOG 06-25 (matrix) · results/20260705_005620_*, 20260708_180117_* (traces)

Part 6 — The questions

Sanford proves the log-depth construction **exists**. My runs say gradient descent does not **find** it on data with no statistical shortcut.

Is there a representability–trainability gap for connectivity, and what closes it?

Every paper I have read is about what a transformer **can represent**, or about **a dataset**. None establishes that training finds the construction.

Thm 2 supplies the premise; my grid plus the capacity control supplies the negative side; the trace result supplies a positive resolution.

A. Encodings as precomputation oracles. Quantify the exchange rate — how many layers does each encoding buy? The Laplacian result is one clean data point; SPD and fixed- n adjacency rows are two more. Directly extends the paper you gave me.

B. A leak-audit protocol for graph-reasoning benchmarks. Six instances across four setups, two of them published. Least risky, most immediately useful — but a methods contribution rather than a theoretical one.

C. Which supervision makes an algorithm learnable? The matrix target and the BFS trace both work where the binary label fails, and there is a pattern in **which** intermediate steps must be made explicit.

Full write-up with evidence status and decisive experiments in `docs/RESEARCH-QUESTIONS.md`

1. **Kill the local bridge-detection shortcut in `connectedness_hard`.** Every negative result I have rests on this dataset.
2. **Re-run the depth $\times$ width grid parameter-matched**, the way Yehudai §6.1 does. Mine accumulated across four sessions with drifting configs and no fixed budget.
3. **Test the width axis properly.** Their claim is $\Omega(n)$ -to- $O(n \text{ polylog } n)$ width at fixed depth; my grid tops out at width 256 with n fixed, so I have never varied width **against** n — the only way their bound is falsifiable.
4. **Write the Yehudai reading companion**, as I have for Sanford and Ye.

Caveats on record: the $\ln 2$ plateau predates my run logger (no results JSON); I never ran `lap_full` inside their pipeline; my reproduction used the supplementary code, which may predate the arXiv v3 I cite.

1. The theory is a **tradeoff**, not a single bound: connectivity costs $O(\log N)$ depth at sub-polynomial width (Sanford), or $O(n \text{ polylog } n)$ width at fixed depth (Yehudai).
2. **Every benchmark I audited leaked** — including two published ones and my own.
3. Laplacian encodings solve it at depth 1, width 8 — confirming a trade-off Yehudai et al. state theoretically, and showing it survives an adversarial dataset. The computation moved into preprocessing; it did not disappear.
4. With the leaks sealed, nothing I trained learned the binary task at any depth or width I ran — and the failure is generalization, not capacity.
5. Changing **supervision** (dense matrix target, or a BFS trace) does work.

Deck source reports/progress-presentation.typ · questions docs/RESEARCH-QUESTIONS.md · day-by-day log docs/CHANGELOG.md